

EllipticBSAMTree Code Documentation
**A strict-tree block-structured adaptive multigrid solver
for cell-centered elliptic problems**

Documentation of the supplied MATLAB implementation
following the Fortran BSAM 2.0 library (SuperBSAM)

July 2026

Contents

Preface	ii
1 Orientation	1
1.1 The solver family and its lineage	1
1.2 What “strict tree” means	1
1.3 Where things live	2
2 The Strict Patch Tree	3
2.1 Two index systems per patch	3
2.2 The single-parent window	4
2.3 Per-parent Berger–Rigoutsos clustering	4
2.4 Proper nesting without restarts	5
3 Single-Parent Transfers in the Elliptic Solver	6
3.1 What changed relative to EllipticBSAM	6
3.2 The duplicated shared-face lesson	6
3.3 Verification	7
4 Verification	8
4.1 EllipticBSAMTree: runAllTreeTests	8
5 Mathematical and Diagnostic Clarifications	9
5.1 Compatibility is a source projection	9
5.2 Reaction fixes the mean	9
5.3 Proof boundaries for the actual QIF stencil	9
5.4 RL environment: what is measured	10
5.5 Headless regression command	10
6 Function Reference	11
6.1 EllipticBSAMTree (ebt*)	11

Preface

EllipticBSAMTree (function prefix ebt) solves $-\operatorname{div}(D\nabla u) + Cu = f$ on a rectangle with Dirichlet/Neumann sides, using the block-structured adaptive FAS multigrid (BSAM) methodology of Feng–Guo–Lowengrub–Wise [1] reorganized around a *strict single-parent patch tree* — the data structure of the Fortran reference library SuperBSAM/BSAM2.0 (treeops.f90 and friends). It is numerically identical to the companion EllipticBSAM solver — same discretization, same QIF/LIF coarse–fine interpolation, same conservative flux corrections — but every fine patch has exactly one parent that fully contains it, which turns every coarse–fine transfer into a single slice of one parent array.

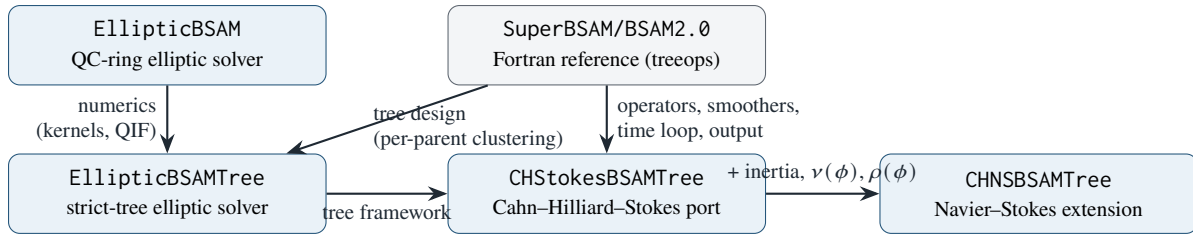
The reader is assumed to know the companion document *BSAM Code Documentation* (in the EllipticBSAM folder), which develops the discretization, the QIF/LIF ghost interpolation, the flux-balance corrections and the AFAS V-cycle in detail. This document concentrates on what is *different*: the tree, its two index systems, per-parent clustering, and the single-parent transfer machinery. The two sibling solvers built on this framework are documented in their own folders: CHStokesBSAMTree (Cahn–Hilliard–Stokes) and CHNSBSAMTree (Cahn–Hilliard–Navier–Stokes).

Chapter 1

Orientation

1.1 The solver family and its lineage

Both packages inherit their numerics from EllipticBSAM and their *architecture* from the Fortran BSAM 2.0 library. The family tree:



1.2 What “strict tree” means

In EllipticBSAM the clusterer works on a whole level at once, so a fine patch may overlap several coarse patches; the per-patch `parents{p}` field is a *list*, and every coarse–fine transfer gathers from a union of coarse patches through precomputed per-cell index lists (the “QC ring” machinery). That design is correct and, as the companion audit showed, carries no convergence penalty — but it is not a tree.

The tree variants instead enforce, exactly as the Fortran does:

Implementation invariant 1.1 (single parent). *Every non-root patch is created by clustering inside one parent patch. Its footprint, recorded in parent-local cell coordinates as `mbounds`, satisfies $1 \leq \text{mbounds} \leq \text{parent.mx}$: a child may touch its parent’s edge but never extend past it, and never spans two parents.*

A physical feature that crosses two coarse patches therefore produces *two abutting children*, one per parent, which communicate as same-level siblings — never one straddling child (figure 1.1).

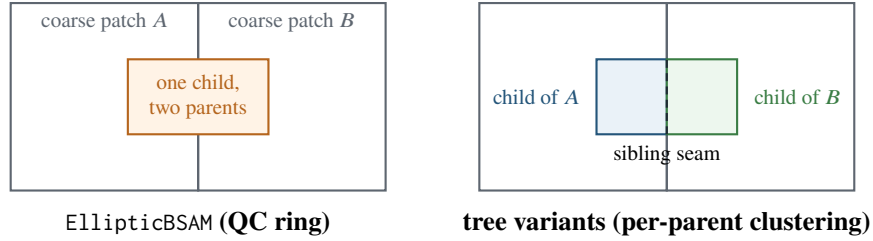


Figure 1.1: A tagged feature crossing two coarse patches. The QC-ring solver creates one multi-parent child; the tree solvers split it per parent, and the two children couple through the same-level (sibling) exchange.

Key idea

The payoff of [implementation invariant 1.1](#) is architectural: the $(c_{m_x}+2) \times (c_{m_y}+2)$ parent window of a child — interior *and* ghost ring — becomes *one contiguous slice of the parent's padded array*. Every coarse–fine transfer (window gather, ring refresh, restriction, FAS loading, correction, coefficient fill-down) is a sliced copy against a single known array. The per-cell multi-patch resolver machinery of the QC-ring design disappears from the hot path.

1.3 Where things live

	EllipticBSAMTree	CHStokesBSAMTree
entry point	ebtSolve	chsRun
problem definition	ebtProblem	chsProblem
options	ebtOptions	chsOptions
test suite	tests/runAllTreeTests (T1–T9)	tests/runChsTests (C1–C6)
tree validator	ebtTreeCheck	chsTreeCheck

Both run headless as `matlab -batch "cd('<pkg>/tests'); <suite>".` Diagnostic messages use the tags `[ebtsam]` and `[chs]`.

Chapter 2

The Strict Patch Tree

2.1 Two index systems per patch

Following the Fortran `nodeinfo` type, every patch carries two coordinate records ([figure 2.1](#)):

- `lev.box(p, :)` = $[i_{lo} \ i_{hi} \ j_{lo} \ j_{hi}]$ — the patch’s cell range in the *global index space of its own level* (the Fortran `mglobal`). It drives everything *horizontal*: owner maps, sibling ghost exchange, tag buffering, plotting.
- `lev.mbounds(p, :)` = $[I_l \ I_h \ J_l \ J_h]$ — the patch’s coarsened footprint in *its parent’s local cell space*, 1-based (the Fortran `mbounds`). It drives everything *vertical*: window/ring slices, restriction and load scatters, coefficient fill-down, correction windows.
- `lev.parent(p)` — the index of *the* parent patch on level $k-1$ (0 on the coarsest level); `lev.children{q}` lists the direct children.

The two systems are locked together: with $q = \text{parent}(p)$ and $qb = \text{box}(q, :)$,

$$\text{mbounds}(p, :) = \left[\frac{i_{lo}+1}{2}, \frac{i_{hi}}{2}, \frac{j_{lo}+1}{2}, \frac{j_{hi}}{2} \right] - [qb_1-1, qb_1-1, qb_3-1, qb_3-1],$$

which requires the global box to be odd-start/even-end — automatic, because a child is the exact $2\times$ refinement of a cluster box. `ebtTreeCheck/chsTreeCheck` verify this identity, the containment of [implementation invariant 1.1](#), pairwise-disjoint siblings, the children/covered bookkeeping, and the ring nesting, on every hierarchy the test suites build.

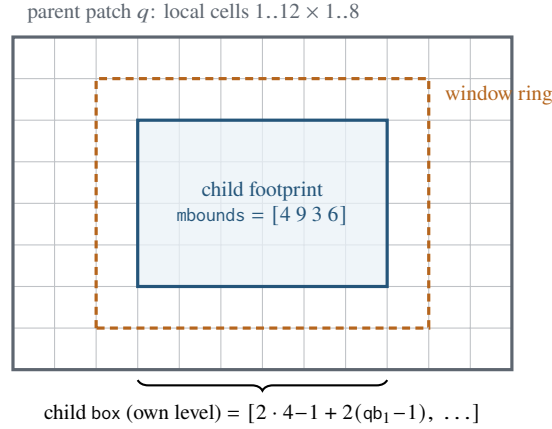


Figure 2.1: The lineage record. The child’s coarsened footprint (mbounds, blue) lies inside its single parent; the window ring (orange) extends one parent cell around it and never leaves the parent’s *padded* array — where it exits the parent’s interior it lands on the parent’s own ghost cells.

2.2 The single-parent window

Let the child have footprint $[I_l..I_h] \times [J_l..J_h]$ in parent cells, $c_{m_x} = I_h - I_l + 1$, $c_{m_y} = J_h - J_l + 1$. Since parent cell (i, j) sits at padded index $(i+1, j+1)$ and $1 \leq I_l, I_h \leq \text{parent.mx}$, the whole window

$$W = \text{parent } Q(I_l : I_h+2, J_l : J_h+2)$$

is a legal slice: rows I_l and I_h+2 are the ring cells I_l-1 and I_h+1 , which are parent *ghost* cells exactly when the child touches the parent’s edge. This is the Fortran InterpolateCoarseFine contract verbatim: *the child reads only parent%*; sibling and physical closures reach the child indirectly, through the parent’s synchronized ghost layer.

Code path

ebtBuildOps stores, per patch: ops.win (the window slice), ops.ring (its four perimeter slices, refreshed into QC before each QIF interpolation), ops.usc/ops.fsc (single-rect scatters of the restricted solution and the FAS load into the parent), ops.pcc/pdew/pdns (parent coefficient slices), and ops.corrW/E/S/N (flux-correction targets). The V-cycle hot path (ebtTransfer, ebtFillGhosts, ebtCoarseLoad, ebtCoeffFilldown) consists of sliced assignments against these descriptors.

2.3 Per-parent Berger–Rigoutsos clustering

Regridding follows the Fortran pipeline (bsamroutines.f90: $\text{AMR} \rightarrow \text{EstimateLevelErrors} \rightarrow \text{GridAdapt} \rightarrow \text{RefineGrid} \rightarrow \text{NewSubGrids}$) with one deliberate simplification (section 2.4):

1. **Tag** on the current finest level, per patch (indicator of choice; the CHStokesBSAMTree documentation for its tagger).
2. **Buffer globally.** Tags are dilated by TagBuffer on the *level-wide* mask, so a buffer that overflows a patch edge lands in the neighboring patch — the Fortran BufferAndList/InflateEdgeTags broadcast, including the periodic wrap where applicable.

3. **Clip for proper nesting** (section 2.4).
4. **Cluster per parent.** Berger–Rigoutsos signature splitting runs *independently on each patch’s own tags* (RefineGrid passes NewSubGrids the single patch’s errorflags); candidate rectangles start from the tags’ bounding box inside the patch and only ever shrink or split, so no cluster box can leave its parent — [implementation invariant 1.1](#) holds by construction.
5. **Refine.** Each accepted box becomes one child (its exact $2\times$ refinement), tagged with its parent’s index.

Only the buffering is global; the clustering is local. As in both parent codes there is *no aspect-ratio constraint* — boxes are free rectangles subject to MinBoxWidth, the fill-ratio target, and the nesting clip.

Implementation note

Per-parent clustering fragments features that cross patch boundaries. Occasionally a fragment is a sliver (one cell wide after the nesting clip) that cannot grow within its parent; such boxes are merged into an adjacent box of the same parent when possible and otherwise dropped with a warning (mergeOrDropThin) — locally *less* refinement, never a fragile patch. The append-only elliptic driver does not revisit dropped cells on older levels; coverage must be checked when resolving a prescribed region is required. The nesting clip keeps the mesh 1-conforming. The Fortran instead grows such boxes over untagged parent cells (its trim floor minimumgridpoints without an allowed-region concept), refining slightly more in these spots.

2.4 Proper nesting without restarts

The Fortran enforces 2-level conformity *a posteriori*: a patch-resolution level map (levellandscape) detects level- L patches adjacent to level- $(L-2)$ regions, and the mesh build restarts from level $L-2$ with augmented tags (FindMeshDefects, up to 100 amrrestarts). The MATLAB tree solvers instead *prevent* defects: tags are clipped to cells whose full 8-neighborhood is covered by the current finest level (erode8; x-periodic in the CH–Stokes solver). Consequences, all verified by the tree checkers:

- every ring cell of a child is covered by the parent level’s patch union (or lies beyond a physical boundary), so the QIF stencil always reads synchronized coarse data;
- interface jumps are one level (1-conforming composite mesh);
- no rebuild loops — each regrid pass is a single sweep.

Chapter 3

Single-Parent Transfers in the Elliptic Solver

EllipticBSAMTree keeps the discretization, kernels, smoother, QIF/LIF interpolation, flux-balance corrections, composite residual and direct coarsest solve of EllipticBSAM *unchanged*. This chapter records only the moving parts that were rebuilt on the tree.

3.1 What changed relative to EllipticBSAM

	EllipticBSAM	EllipticBSAMTree
clustering	level-global B–R	per-parent B–R (section 2.3)
lineage	parents{p} list	parent(p) scalar + mbounds
window/ring	per-cell gathers from all owners	one padded-parent slice
restriction, FAS load	scatters over all intersecting owners	single-rect scatter into the parent
coefficient fill-down	rectIsect over all coarse patches	sliced write into the parent + shared-face push (section 3.2)
correction targets	owner-map scatter	unchanged (targets may be the parent’s siblings)

The flux-balance correction targets are the one place the tree still routes through an owner map: when a child touches its parent’s edge, the coarse cell just outside the footprint belongs to a *sibling of the parent*. This is precomputed (ops.corrW/E/S/N), the static analogue of the Fortran’s global masscorrlayer strip list.

3.2 The duplicated shared-face lesson

Implementation note

Coarse patch boundary *faces* are stored twice — once in each abutting patch’s face-coefficient array. When a child’s footprint touches its parent’s edge, the coefficient fill-down (coarse face $D \leftarrow$ mean of the two overlying fine faces) must write the averaged value into *both* copies, or the two sides of that face compute different fluxes and the discrete conservation identity breaks. ebtCoeffFilldown pushes boundary-face writes to the abutting siblings through the owner map (pushFaceX/pushFaceY). The symptom when this is missing is instructive: a mass defect that *floors* (does not shrink with tighter solver tolerance) and appears only for variable D on adaptive meshes — constant coefficients hide it, because the fill-down average then equals the analytic value.

3.3 Verification

tests/runAllTreeTests reruns the entire EllipticBSAM suite (T1–T8) on the tree solver and adds T9:

- **T1–T7 parity.** Identical numbers to the QC-ring solver: second-order L^∞ rates (1.98–2.03) on uniform, box-in-box, L-shaped and fully adaptive hierarchies for QIF and LIF; h - and depth-independent V(3,3) factors 0.026–0.054; conservation defects $7 \cdot 10^{-17}$ – $3 \cdot 10^{-15}$.
- **T8 robustness.** 24 random tag disks; per-parent clustering yields 50 patches (QC-ring: 49) of minimum width 4, factor 0.052, mass defect $1.1 \cdot 10^{-13}$.
- **T9 tree invariants.** ebtTreeCheck passes on the adaptive hierarchy, and the engineered straddling scenario (a tag centered on the shared edge of two coarse patches) produces exactly two children with parents [1 2] — never a multi-parent child — with V-cycle factor 0.033 and mass defect $9.5 \cdot 10^{-13}$.

Remark 3.1. *The companion audit measured identical per-cycle factors for a multi-parent child (QC ring) and a single-parent child on the same mesh. The tree buys structure and simplicity, not V-cycle speed; its performance value lies in the $O(1)$ sliced transfers and in what it enables next — the CH–Stokes solver’s face-centered machinery would be substantially harder against multi-parent children.*

Chapter 4

Verification

4.1 EllipticBSAMTree: runAllTreeTests

T1–T8 reproduce the EllipticBSAM suite on the tree ([section 3.3](#)); T9 validates the tree invariants and the per-parent splitting of a straddling feature. Headline numbers (MATLAB R2026a): uniform and adaptive L^∞ rates 1.98–2.03; V(3,3) factors 0.026–0.054 independent of h and depth; conservation defects at roundoff (10^{-16} – 10^{-13}); suite time ≈ 37 s.

Chapter 5

Mathematical and Diagnostic Clarifications

5.1 Compatibility is a source projection

For all-Neumann data and $C \equiv 0$, the discrete source and the outward boundary derivative must satisfy

$$\sum_{i \in \mathcal{V}} h_{x,i} h_{y,i} f_i + \sum_{F \subset \partial\Omega} A_F D_F g_F = 0.$$

`ebtCompatProject` subtracts the defect divided by the visible domain area from the original sampled source on every level. The removed constant is `H.compatibility.sourceShift`; inspect it because incompatible data are thereby changed. Per-level `sourceShift` bookkeeping ensures that previously projected coarse data and fresh fine data are treated consistently at every refinement. Higher source quadrature improves the source integral; it is not exact for arbitrary data and does not by itself enforce the boundary-flux compatibility condition.

5.2 Reaction fixes the mean

Since $\mathcal{L}(u + a) = \mathcal{L}u + Ca$, only the all-Neumann, $C \equiv 0$ problem has an additive-constant gauge. The error routine aligns the error mean exactly in this singular case, for every `SourceQuad`; otherwise it retains the absolute error. The removed shift is `err.meanShift`. A small positive reaction is not classified as zero using an absolute numerical threshold. `testEbtNeumannSemantics` verifies this rule and repeated refinement of a compatible quartic manufactured solution.

5.3 Proof boundaries for the actual QIF stencil

The tree changes data movement, not the stencil coefficients. At a regular west QIF face,

$$Q_{1,3} = \frac{8}{15} Q_{1,2}^c + \frac{2}{3} Q_{2,4} - \frac{1}{5} Q_{3,5}.$$

The equation for $Q_{2,3}$ therefore contains $+DQ_{3,5}/(5h_x^2)$. This is a positive off-diagonal coefficient even for constant D on a square grid, since $Q_{3,5}$ is not an ordinary nearest neighbor of $Q_{2,3}$. Blanket Z -matrix and M -matrix claims for the fully eliminated multidimensional QIF operator are consequently unjustified. Conservation follows from matching face fluxes; a comparison principle, energy estimate and uniform V-cycle

bound need separate arguments. The companion theory document’s multidimensional sign lemma and conclusions depending on it remain proof-review items.

The physical boundary fold gives exact point updates with neighbors fixed. At coarse–fine interfaces, ghost interpolation is lagged within each color pass; global parity alone does not make this literal Gauss–Seidel on the fully eliminated, wider QIF stencil. The reported contraction factors establish performance on the tested mesh families.

5.4 RL environment: what is measured

The `r1/` API can observe an unsolved root (`SolveRoot=false`), take individual cycles, and refine or polish the current hierarchy. A stage observation supersedes the residual of any preceding single cycle. `testR1State` verifies these transitions and rejection of non-finite or fractional box indices.

The reward’s relative-truncation-error proxy omits a two-cell margin around every patch and all patches narrower than eight cells. A smaller proxy can therefore partly reflect a changed measurement region. It is not a guaranteed discretization-error estimator; report true manufactured error, residual, visible cells and work alongside it. For noisy numeric forcing, comparison with the clean generating field includes data/interpolation error and is not error against an exact solution of the perturbed PDE. Calibration-fit correlations describe the sampled calibration set, not an out-of-sample error guarantee.

5.5 Headless regression command

```
1 /Applications/MATLAB_R2026a.app/bin/matlab -batch "maxNumCompThreads(2); cd('EllipticBSAMTree/tests');
    unitTreeKernels; runAllTreeTests; stressTreeInterfaces; testR1Api; testR1State;
    testEbtNeumannSemantics"
```

Keep the tree tests on an isolated MATLAB path to avoid the other package’s `mmsCases`. Historical positive-reaction Neumann error tables may use a mean-aligned error; rerun before comparing with the corrected absolute norms. Dated audit logs preserve both versions.

Chapter 6

Function Reference

6.1 EllipticBSAMTree (ebt*)

function	responsibility
ebtSolve	driver: setup, solve–tag–refine loop, outputs
ebtProblem, ebtOptions	problem and option structs
ebtSetup, ebtMakeLevel	hierarchy and per-patch fields (+ lineage)
ebtAddLevel	append a level from per-parent cluster boxes
ebtTagCluster	tagging, global buffer, nesting clip, per-parent B–R
ebtBuildOps	owner maps, same-level exchange, single-parent descriptors
ebtFillGhosts, ebtInterpCf	ghost priority order; QIF/LIF stencils
ebtTransfer	restrict / ring / correct / filldown
ebtVcycle, ebtSolveCycles, ebtRelaxLevel	AFAS engine
ebtCoarseLoad, ebtFaceCorrections	FAS load + flux-balance corrections
ebtCoeffFilldown, ebtParentCoeffs,	coefficient compatibility
ebtGatherParentCoeffs	
ebtCoarsest, ebtKernels, ebtBcFold	level-1 solve; kernels; boundary fold
ebtCompResidual, ebtErrorNorms, ebtMassCheck	diagnostics
ebtTreeCheck	tree invariant validator
ebtPlot, ebtSample, ebtRte, ebtCompositeMean,	utilities
ebtGatherWindow	

Bibliography

- [1] W. Feng, Z. Guo, J. S. Lowengrub, S. M. Wise, *A mass-conservative adaptive FAS multigrid solver for cell-centered finite difference methods on block-structured, locally-cartesian grids*, J. Comput. Phys. **352** (2018) 463–497.
- [2] M. J. Berger, I. Rigoutsos, *An algorithm for point clustering and grid generation*, IEEE Trans. Systems, Man, and Cybernetics **21** (1991) 1278–1286.
- [3] S. P. Vanka, *Block-implicit multigrid solution of Navier-Stokes equations in primitive variables*, J. Comput. Phys. **65** (1986) 138–158.